

Práctica final RA1



Implantación de Aplicaciones Web (0376) · ASIR 2º · Sesión 7

Misión

El dueño de un salón recreativo quiere una web con el ranking de récords de su arcade. Tu trabajo es montar el servidor (Apache + PHP + MariaDB en contenedores), asegurarlo, desplegarlo en local y documentarlo.

Cada nivel superado te da una moneda 🪙 (un código). Anota las monedas en tu `README.md`.

Se evalúa con una lista de cotejo: cada ítem está superado o no, así que léete el enunciado entero antes de empezar.

Puedes reutilizar todo lo trabajado de las prácticas S1 a S6 o crearte uno nuevo limpio.

Material que te doy

```
salon-recreativo/
├── docker-compose.yml  (tu stack de las sesiones 1-6, lo adaptarás)
├── .env                (lo creas tú, te doy un ejemplo; NO se sube a GitHub)
├── .gitignore          (te lo doy)
├── Dockerfile          (lo creas tú en el nivel 2)
├── db/init.sql         (te lo doy; no lo modifiques)
├── src/index.php       (te lo doy; no lo modifiques)
└── README.md           (lo escribes tú)
```

Nivel 1 · Arranque de la base de datos (guiado)

1.1 Variables de entorno

Crea un fichero `.env` en la raíz del proyecto (copia `.env.example` y rellena las contraseñas; elige dos distintas):

```
MARIADB_ROOT_PASSWORD=<elige una contraseña>
MARIADB_DATABASE=arcade
MARIADB_USER=jugador
MARIADB_PASSWORD=<elige otra>
```

⚠ Comprueba que `.env` está en `.gitignore` **antes de tu primer commit**. Si ya lo subiste sin querer, avisa al profesor: borrarlo del último commit no lo borra del historial.

1.2 Servicio de la base de datos

Añade a tu `docker-compose.yml` el servicio `db` (si ya tienes uno de la sesión 4, adáptalo).

Completa los `\_\_\_`:

```
db:
  image: mariadb:___          # una versión concreta, NO "latest"
  env_file: .env
  volumes:
    - ___:/var/lib/mysql      # volumen con nombre: los datos persisten
    - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  healthcheck:
    test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
    interval: 5s
    retries: 10
  networks: [___]           # la misma red que el servicio web * lo explico en clase *
```

Declara el volumen en la sección `volumes`: del final del fichero.

Después, en el servicio `web`, pasa **solo** las variables que necesita la aplicación y espera a que la BD esté sana:

```
web:
  environment:
    DB_HOST: db
    DB_NAME: ${MARIADB_DATABASE}
    DB_USER: ${MARIADB_USER}
    DB_PASS: ${MARIADB_PASSWORD}
  depends_on:
    db:
      condition: service_healthy
```

 **Piensa: ¿por qué no le pasamos al servicio web la contraseña de root de la base de datos (env_file: .env)? Anota la respuesta en el README.**

1.3 Arranque y comprobación

```
docker compose up -d --build
docker compose ps          # db debe aparecer como "healthy"
docker compose exec db mariadb -u jugador -p arcade
```

Ya dentro del cliente SQL (te pedirá la contraseña de `jugador`):

```
SHOW TABLES;
SELECT * FROM ranking;
SHOW GRANTS;
```

Copia en el README el resultado de `SHOW GRANTS` y contesta: ¿sobre qué base de datos tiene permisos el usuario `jugador`? ¿Por qué no usamos `root` desde la aplicación?

1.4 Persistencia

Ejecuta `docker compose down` y luego `docker compose up -d`: comprueba que los datos siguen ahí. Después prueba `docker compose down -v` y vuelve a levantar.

Anota en el README qué diferencia hay y por qué.

 **Moneda 1:** En la tabla **ranking** hay una fila cuyo jugador es una moneda (no sale en la web). Encuéntrala con el cliente SQL y cópiala en el README.

(antes del punto 2)¿qué es un Dockerfile?

Un **Dockerfile** es un fichero de texto con las instrucciones para construir una **imagen propia** a partir de una imagen oficial. Lo usamos cuando la imagen oficial se queda corta (por ejemplo, le falta una extensión).

- **Dockerfile** → cómo se **construye** una imagen (build).
- **docker-compose.yml** → cómo se **ejecutan** varios servicios juntos (up).

Ejemplo (instala la extensión bcmath y copia una carpeta):

```
FROM php:8.3-apache
RUN docker-php-ext-install bcmath
COPY src/ /var/www/html/
```

Instrucción	Qué hace
FROM	Imagen de partida
RUN	Ejecuta un comando al construir la imagen
COPY	Copia ficheros de tu proyecto dentro de la imagen

Cómo se conecta con el compose. En el servicio, build: . sustituye a image::

```
web:
  build: .
  ports:
    - "8080:80"
```

Cuidado. Si cambias el Dockerfile, reconstruye con `docker compose up -d --build`. Si no, Docker reutiliza la imagen anterior y parecerá que no hace caso.

¿Por qué no instalar la extensión a mano dentro del contenedor? Porque se perdería al recrearlo. Lo que está en el Dockerfile es reproducible y queda versionado en GitHub.

Nivel 2 · Conexión desde PHP (semiguiado)

El `index.php` que te doy se conecta a MariaDB con `mysqli`, pero la imagen `php:8.3-apache` no trae esa extensión. Si abres la página ahora verás un error: es lo esperado.

2.1 Crea un **Dockerfile** en la raíz del proyecto, basándote en la píldora, que:

- parta de `php:8.3-apache`,
- instale la extensión `mysqli`,

- copie `src/` a `/var/www/html/`.

2.2 Cambia el servicio web del compose para que use `build: .`, reconstruye y abre `http://localhost:8080`.

2.3 La página muestra el ranking, la hora del servidor (PHP) y la hora del navegador (JS). En el README explica **qué se ejecuta en el servidor y qué en el cliente**, y por qué las dos horas podrían no coincidir.

 **Moneda 2: Aparece en la página cuando la conexión funciona. Cópiala en el README.**

Nivel 3 · Seguridad (autónomo)

Comprueba, arregla si hace falta y **documenta en el README** (con el comando y su salida):

- **Sin credenciales en el repositorio.** `git ls-files` no debe listar `.env` (solo `.env.example`), y `git log -p | grep -F "<tu contraseña>"` no debe devolver nada.
- **Puerto 3306 no publicado** en el host: se ve con `docker compose ps`.
- **La aplicación usa `jugador`, no `root`.**
- **Apache y PHP no revelan su versión.** Comprueba las cabeceras con `curl -I http://localhost:8080`: no debe aparecer ni el número de versión de Apache ni `X-Powered-By`.

 **Pista** para lo último: busca las directivas `ServerTokens`, `ServerSignature` y `expose_php`. Puedes crear ficheros de configuración desde el Dockerfile con `RUN printf ... > fichero`.

 **Moneda 3: te la doy yo en directo cuando compruebe `curl -I` y `docker compose ps` en tu equipo.**

Nivel 5 · Documentación

Tu `README.md` debe incluir:

- Descripción del stack y las **tecnologías** (servidor web, SGBD, intérprete, qué corre en cliente y qué en servidor).
- **Diagrama de arquitectura en Mermaid** (GitHub lo dibuja solo, puedes preguntarle a la IA).
- Punto de partida:

```
graph LR
  N[Navegador] -->|8080| W[Apache + PHP]
  W -->|red Docker| B[(MariaDB)]
```

- **Comandos de despliegue** (local y en Play with Docker).
- Las **respuestas** a las preguntas de los niveles 1 y 2 y las comprobaciones del nivel 3.
- Las **3 monedas**.
- Apartado «**Problemas que me encontré y cómo los resolví**» (mínimo uno).

 **Pista :** `init.sql` solo se ejecuta cuando el volumen de la BD está **vacío**.

Si lo cambias y no ves efecto, necesitas `docker compose down -v` y volver a levantar.

Corrección

Cuando acabes sube el readme a AULES y llama a Ana para que te corrija y pase el check .